Lab 2 Section 3

Team Silver

Old Dominion University

CS 411W

Thomas J. Kennedy

27 October 2026

Version 1

Table of Contents

3 Specific Requirements 3
3.1 Functional Requirements 3
3.1.1 User Interface 3
3.1.2 User Authentication and Access Control 4
3.1.3 Order and Deal Management 5
3.1.4 Inventory Management 6
3.1.5 Production Management 7
3.1.6 Logistics Management 8
3.1.7 Document Management 9
3.1.8 Analytics and Reporting 9
3.1.9 Backend and Data Processing 10
3.2 Performance Requirements 11
3.2.1 Scalability 11
3.2.2 Availability 12
3.3 Assumptions and Constraints 12
3.4 Design Constraints 12
3.4.1 Development Language & Technologies 13
3.4.2 Demonstration Format 13
3.4.3 Dependency and Deployment Management 13

3.4.4 Build System....13

3.5 Other Requirements....13

3.5.1 Database....14

3.5.2 Security....17

3.5.3 Maintainability....17

3.5.4 Reliability....17

Listing of Figures

No table of figures entries found.

Listing of Tables

No table of figures entries found.

3 Specific Requirements

3.1 Functional Requirements

The BevOS system is designed to help beverage manufacturing, and co packaging businesses manage their daily operations through a centralized web application. The system will allow users to access and manage information related to deals, orders, inventory, production, logistics, and business documents.

The following requirements describe how users will interact with BevOS and the functions that each major component of the system is expected to provide.

3.1.1 User Interface (O: Beayon)

The BevOS system should provide a web-based interface that allows authorized users to navigate and interact with the platforms available operational features.

3.1.1.1 Navigation Menu

The system should provide a navigation menu that will allow users to move between pages.

- The navigation menu shall provide links to the main sections of the application.
- Selecting a navigation link shall direct the user to the corresponding page.
- The navigation menu shall remain useable when the browser window is resized.

3.1.1.2 Dashboard

The dashboard should serve as a central location for users to access operational information within BevOS.

- The system should provide a dashboard for viewing available business information.
- The dashboard shall organize information into clearly identified sections.
- Users shall be able to navigate from the dashboard to the corresponding operational pages.

- The dashboard shall display information retrieved from the backend if available.
- When information cannot be retrieved for the dashboard, the system shall notify the user.

3.1.1.3 Deals Page

The Deals page shall provide users with a centralized view of deal information stored in BevOS.

- The system shall retrieve available deal records from the backend.
- The deals page shall display the information associated with each returned record.
- The system shall support displaying multiple deal records.
- When no deal records are available the page shall display a message indicating that no records were found.
- If the request for deal information fails, the system shall display an error message.
- The page shall update its displayed information after a successful request for current deal data.

3.1.2 User Authentication and Access Control

BevOS shall provide an authentication system to identify users and control access to business information. Only authenticated users shall be permitted to perform operations that require authorization.

3.1.2.1 User Login (O: Fernandes)

- The system should provide a login page where users can enter their credentials.
- The login page shall contain separate fields for the user's account identifier and password.
- Passwords shall be hidden while being entered.
- The system shall submit the entered credentials to the backed for validation.

- Users who provide valid credentials shall be granted access to the appropriate application features.
- If authentication fails, the system shall notify the user and allow another login attempt.

3.1.2.2 User Sessions (O: Fernandes)

- The system shall maintain an authenticated session for users who successfully log in.
- The system shall allow authenticated users to log out.
- When a user logs out, the system shall end the active session.
- The system shall prevent unauthenticated users from accessing protected operations.

3.1.2.3 User Permissions (O: Fernandes)

- The system shall determine which operations a user is authorized to perform.
- Users shall only be permitted to modify information when their assigned permissions allow it.
- The system should reject attempts to perform unauthorized operations.
- Administrative functions shall be restricted to users with the appropriate permissions.

3.1.3 Order and Deal Management (O: Beayon)

BevOS shall provide functionality for organizing and tracking business transactions. This component will help users maintain a consistent record of orders and deals without relying entirely on spreadsheets or email conversations.

3.1.3.1 Order Creation

- Authorized users shall be able to create a new order record.
- The system shall provide fields for entering the information required to identify and process an order.

- The system shall validate required information before accepting an order.
- After successful submission the system shall save the order and make it available for later retrieval.

3.1.3.2 Order Tracking

- The system shall allow users to view existing orders.
- Each order shall have an identifiable record.
- Authorized users shall be able to update an order status.
- Users shall be able to view current order status.
- Changes to an order shall be reflected when updated.

3.1.4 Inventory Management

The inventory management component shall provide users with a way to maintain records of materials and products used in beverage operations. This will help the business keep information organized as materials are received, used, or prepared for production.

3.1.4.1 Inventory Records

- The system should maintain records for inventory items.
- Each inventory record shall contain an identifier and the information needed to describe the item. Authorized users shall be able to add new inventory records.
- The system shall provide a way to view existing inventory information.

3.1.4.2 Inventory Updates

- Authorized users shall be able to update the recorded quantity of an inventory item.
- The system shall validate quantity values before saving an update.
- Changes to inventory records shall be saved in the database.

- The system shall display the most recently saved quantities.

3.1.4.3 Inventory Availability

- Users shall be able to view the recorded quantity of an inventory item.
- The system shall distinguish between items with available inventory and items with no inventory.
- The system shall prevent users from submitting invalid quantity values.

3.1.5 Production Management

The Production Management component shall allow users to organize and monitor production related information. Its whole purpose is to provide a consistent record of manufacturing activity and make it easier to determine the current state of production work.

3.1.5.1 Production Records

- Authorized users shall be able to create production records.
- Each production record shall contain an identifier and the relevant production information.
- The system shall associate production records with their related business orders when applicable.
- Users shall be able to retrieve and view existing production records.

3.1.5.2 Production Status

- The system shall maintain a status for each production record.
- Authorized users shall be able to update the status of production work.
- Users shall be able to view the current status of a production record.
- Status changes shall be saved and reflected in subsequent request for that record.

3.1.5.3 Production Information

- The system shall provide a way to record relevant production details.
- Authorized users shall be able to update production information as work progresses.
- The system shall validate required fields before saving changes.
- The system shall make the updated information available to users with appropriate access.

3.1.6 Logistics Management

The Logistics Management component shall help users organize shipment and delivery information associated with business operations. The goal is to give employees a central location for checking and updating the status of outgoing or incoming shipments.

3.1.6.1 Shipment Records

- Authorized users shall be able to create shipment records.
- Each shipment shall have an identifier.
- The system shall store relevant shipment information, including its associated order when applicable.
- Users shall be able to view recorded shipments.

3.1.6.2 Shipment Status

- The system shall maintain a status for each shipment.
- Authorized users shall be able to update shipment statuses.
- Users shall be able to view the current recorded status of a shipment.
- Changes to shipment information shall be saved for future reference.

3.1.7 Document Management (O: Fernandes)

The Document Management component shall provide a central location for storing and accessing documents related to business operations. This will help employees locate relevant information without searching through separate email conversations or local files.

3.1.7.1 Document Upload

- Authorized users shall be able to upload documents.
- The system shall have an identifiable record for each uploaded document.
- The system shall tell user if upload fails
- Documents that are successfully uploaded will be available for later use.

3.1.7.2 Document Retrieval

- Users with authorization shall be able to view document records.
- All documents associated with a particular business record shall be accessible.
- The system shall display a message if a requested document cannot be retrieved.

3.1.7.3 Document Updates

- Authorized users shall be able to update document information.
- The system shall prevent unauthorized changes to saved documents.
- Changes to documents records shall be saved for future retrieval.

3.1.8 Analytics and Reporting (O: Fernandes, M1: De Guzman)

The analytics component shall provide users with a way to review information collected through BevOS. Its purpose is to help employees and management understand business activity using the records already maintained by the system.

3.1.8.1 Analytics Dashboard

- The system should provide a dedicated page for displaying available business analytics.
- Analytics information shall be obtained from records maintained within BevOS.
- The page shall display available results in a readable format.
- The system should notify users when analytics information cannot be retrieved.

3.1.8.2 Operational Summaries

- The system shall support summaries of operational information stored in the application
- Summaries shall use existing records rather than requiring users to re-enter same information.
- The system should clearly identify all the information represented in each summary.
- When no records are available for a summary, the system shall display an appropriate message.

3.1.9 Backend and Data Processing (O: De Guzman, M1: Fernandes)

The BevOS backend shall process requests from the user interface and manage communication with the database. It should support the exchange of information between application components while maintaining the stored records.

3.1.9.1 API Communication

- The backend shall provide endpoints for retrieving supported operational information.
- The system shall process requests submitted through the user interface.
- The backend shall return responses in a format that the frontend can interpret.
- When a request fails, the backend shall provide an appropriate error message.

3.1.9.2 Data Retrieval

- The system shall retrieve requested information from the database.
- Retrieved records shall be returned to the appropriate application component.
- The backend shall provide a way to identify individual records.
- The system shall return an appropriate response when a requested record does not exist.

3.1.9.3 Data Modification

- Authorized operations shall be able to create or update supported records.
- The backend shall validate required information before saving.
- The system shall notify the requested component if the modification was successful or not.
- The system shall prevent invalid requests from modifying stored business records.

3.2 Performance Requirements (O: Fernandes)

This BevOS prototype focuses on functionality and implementation more than runtime performance but some long-term focus is put on Scalability and Availability/Uptime, particularly in regard to preserving information security, integrity, and unified analysis as the distributed network of devices feeding information on Logistics and Production processes changes over time.

3.2.1 Scalability (O: Fernandes)

BevOS is intended to run as a background service/daemon or API endpoint on multiple devices and within various networks to gather and report information to the central database. As this network of distributed devices can change over time an emphasis is put on ensuring the system can gracefully handle the dynamic addition of up to 100 peers in the workflow oversight network.

3.2.2 Availability (O: Fernandes)

Availability and the preservation of information integrity is of paramount
concern to the primary function of BevOS. In addition to aiming for 24/7 availability the system must implement a form of redundancy in the cases of information read/write (to ensure data isn't lost before it is written to disk.

3.3 Assumptions and Constraints (O: Kirby)

- Internet access is required to access and use the BevOS web-based prototype.
- It is assumed that users have access to a compatible web browser and a device capable of running the application.
- It is assumed that the prototype will focus exclusively on workflows within the beverage manufacturing industry.
- The prototype will use synthetic data to simulate logistics and production related activities.
- The prototype will implement the primary workflow functions, while non-target modules may be represented by stubs.
- Document management functionality will be limited to its technical implementation within the prototype.
- Scalability, legal considerations, and advanced security requirements will not be fully explored within the scope of the prototype.

3.4 Design Constraints

This section describes 'off-limits' areas that restrict how the system may be developed.
For BevOS this is driven by the technical requirements and limits of the CS 411W

Course limiting the system to a prototype. The focus is on the web app as a proof-of-concept with some consideration of the novel API, DB, and Distributed Workflow features of BevOS.

3.4.1 Development Language & Technologies

In line with the focus on a proof-of-concept prototype the system is developed in Python with Django as the web app framework. While Django is quite 'heavy' in capability it adopts Python's "batteries-included" philosophy making it ideal for this use case.

3.4.2 Demonstration Format

As a complete system is not expected the BevOS deliverable to be focused on will be prototype demonstration in teleconference interactions. Focus will be on confirmation of system functionality development both in implementation and the supporting verification and validation of such.

3.4.3 Dependency and Deployment Management

The system makes use of Docker containerization for ease of deployment and some approximation of the greater distributed workflow monitoring network design of BevOS but for the purposes of the designed system only the central web app deployment will be considered with stubbing and test data simulation of the network activity.

3.4.4 Build System

The BevOS system is deployed through Docker Compose orchestration of four containerized services: backend, frontend, migrate, and db for the web app and PostgreSQL database components.

3.5 Other Requirements (O: De Guzman)

This section outlines the database schema and functionality, security, maintainability and reliability requirements.

3.5.1 Database

The database will store information for the deals, the clients and suppliers, the commercial records on both sides of a transaction, the documents, and the tracking records. BevOS should be built using PostgreSQL. Each table shall use an integer primary key called id.

3.5.1.1 Clients and Suppliers (O: De Guzman)

The client and supplier tables will store the master record for each party that buys from or sells to the company. A client and a supplier shall each be identified using a unique integer ID. The database shall store the client's name and the supplier name as text, each enforced as unique. The database shall store contact name, email, and phone as text. The database shall store address, billing contact, and payment terms as text. The database shall store notes as text. Clients and suppliers shall remain separate tables.

3.5.1.2 Statuses (O: De Guzman)

The status table will store the states a record is permitted to occupy. A status shall be identified using a unique integer ID. The database shall store a status name as text, enforced as unique. The database shall store a category as an enumerated value from deal, customer_po, customer_invoice, supplier_po, and supplier_invoice. The database shall store a sort order as an integer. The database shall store an active flag as a boolean.

3.5.1.3 Deals (O: De Guzman)

The deal table will store the central record that links all operational and financial data. A deal shall be identified using a unique integer ID. The database shall store a deal number as text, enforced as unique, and a deal name as text. The database shall store a foreign key to one client, one status from the deal category, and one user who created the deal. The database shall store a target date as a date. The database shall store notes as text and created and updated timestamps.

One deal shall relate to many packaging requirements, specifications, commercial records, documents, notes, and status history entries.

3.5.1.4 Packaging Requirements and Specifications (O: De Guzman)

These tables will store what is to be manufactured. Each shall be identified using a unique integer ID and shall store a foreign key to one deal. The database shall store the packaging item type as an enumerated value restricted to can, end, carton, and tray. The database shall store the quantity as a decimal and the unit and unit of measure as text. The database shall store the specification name, value, and unit as text.

3.5.1.5 Customer-Side Records (O: De Guzman)

The customer quote, customer purchase order, and customer invoice tables will store revenue owed to the company by clients. Each shall be identified using a unique integer ID and shall store a foreign key to one deal and one client. Each shall store its document number as text, enforced as unique. The database shall store the document date, and the valid-until, expected-ship, or due date as applicable. The database shall store the total amount as a decimal and the currency as text. The database shall store a status, notes, and created and updated timestamps.

3.5.1.6 Supplier-Side Records (O: De Guzman)

The supplier quote, purchase order, and supplier invoice tables will store costs owed by the company to suppliers. Each shall be identified using a unique integer ID and shall store a foreign key to one deal and one supplier. Each shall store its document number as text, enforced as unique. The database shall store the document date, and the valid-until, expected-delivery, or due date as applicable. The database shall store the total amount as a decimal and the currency as text. The database shall store a status, notes, and created and updated timestamps.

3.5.1.7 Documents (O: De Guzman)

The document table will store the classification and location of uploaded files, with file bytes held on a persisted volume rather than in a column. A document shall be identified using a unique integer ID. The database shall store a foreign key to one deal and a foreign key to the uploading user. The database shall store a module type as an enumerated value from deal, client_po, client_invoice, supplier_po, supplier_invoice, quote, module_item, and general. The database shall store the file name, file path, and file type as text and the file size as a big integer. The database shall store a description as text and an upload timestamp. A document shall be attachable to any record within a deal.

3.5.1.8 Users (O: De Guzman)

The user table will store the information used to authenticate a person and to determine what that person may do. A user shall be identified using a unique integer ID. The database shall store the full name as text. An email address shall be stored as text, enforced as unique, and shall identify exactly one user. The database shall store the password as a salted, one-way hash. The database shall store a role as an enumerated value and an active flag as a boolean.

3.5.1.9 Status History, Notes, and Activities (O: De Guzman)

These tables will store the manual record of what happened to a deal and who recorded it. Each shall be identified using a unique integer ID and shall store a foreign key to one deal. A status history entry shall store a foreign key to the status now held and to the user who changed it, and shall store the changed time. The database shall store notes as text on a status history entry, the note body as text on a deal note with its author, and the activity type and description as text on a deal activity with its performer and performed time. A status history entry shall be append-only and shall never be edited or deleted.

3.5.2 Security (O: Fernandes, M1: De Guzman)

BevOS will require users to enter a password to access their account. To access an account the user shall enter an email address and a password, and the email address shall be unique across all accounts. The user's password shall be encrypted and shall never be stored or transmitted in plaintext. Two access levels are provided. The first permits manual entry and retrieval of deals, commercial records, and documents within the user's role. The second permits administration of master data, statuses, and user accounts. Access control shall be enforced on the server, not only in the interface. Data in transit shall use HTTPS, and database access shall use a dedicated PostgreSQL role with rights on the application schema only.

3.5.3 Maintainability (O: De Guzman)

BevOS will use proven website development software to ensure that the platform is operational on a consistent basis. Allow easy update and maintenance of system servers using Docker and Docker Compose scripted server deployment and management. All schema change shall be made through version-controlled migrations, and direct edits to a live database are prohibited. Shared behavior, namely timestamps and money fields, shall be defined once and reused. The test suite shall run from one command and shall gate integration through GitHub Actions. The development team owns the application, migrations, and pipeline; on handover, administration passes to the company.

3.5.4 Reliability (O: De Guzman)

Access is required 24/7, since clients operate across time zones and an order or status change shall be able to be submitted at all times. The product shall complete 99 percent of transactions without error or kickback, and 100 percent of committed data shall survive a restart. Because the system is offered continuously, all timestamps shall be stored in Coordinated Universal Time and

rendered in the requesting user's local time, so that an approval entered in one zone reads correctly in another. Authentication, deal state, status history, financial records, the database, and the dashboard are critical and shall always be available. The dashboard is critical because continuous operational visibility is the reason the platform exists; a user who cannot see current deal, approval, and delivery status cannot make the decisions the system was built to support, and a stale figure invites a decision made on false information. Reporting exports and the notes feed are non-critical and may be non-functional without affecting critical performance.